Neural Successive Cancellation Decoding of Polar Codes for the Two-User Gaussian Multiple Access Channel

Comprehensive Project Report

Project: EE Master's Thesis — Neural SC Decoder for MAC Polar Codes **Date**: March 30, 2026
Repository: https://github.com/eitanspi/polar-codes-mac

---

1. Introduction and Motivation

1.1 Polar Codes

Polar codes, introduced by Arikan in 2009, are the first provably capacity-achieving codes for symmetric binary-input memoryless channels with explicit construction and efficient encoding/decoding. The key idea is **channel polarization**: by applying a recursive linear transformation F^{otimes n} to N = 2^n independent copies of a channel W, the resulting synthetic channels polarize — some become nearly perfect (capacity -> 1) while others become nearly useless (capacity -> 0). Information bits are placed on the good channels, frozen bits (set to 0) on the bad ones.

1.2 Multiple Access Channel (MAC)

In the two-user MAC, two independent transmitters (User U and User V) send messages simultaneously through a shared channel. The receiver observes a combined output Z = f(X, Y) + noise and must decode both messages. The MAC capacity region defines the set of achievable rate pairs (R_u, R_v).

Onay (ISIT 2013) showed that polar codes achieve the entire MAC capacity region through successive cancellation decoding with an appropriate decoding order (path). Ren, Li, and Olmos (2025) extended this to efficient O(N log N) decoding using computational graphs for all monotone chain paths.

1.3 Project Goal

Develop a **neural network-based SC decoder** for the two-user MAC that: 1. Replaces all analytical decoder operations with learned neural networks 2. Matches or approaches the analytical SC decoder's block error rate (BLER) 3. Scales to practical code lengths (N >= 256)

The neural decoder would enable deployment on channels where the analytical transition probabilities are unknown or too complex to compute.

---

2. Channel Models

2.1 Binary Erasure MAC (BEMAC)

The simplest MAC model. Both users transmit binary X, Y in {0,1}, and the receiver observes Z = X + Y in {0,1,2}.

- **U marginal channel**: W_1(z|x) = sum_y 0.5 * W(z|x,y). Bhattacharyya parameter Z_0_u = 0.5.

- **V conditional channel** (given X): W_2(z|y;x) = W(z|x,y) = delta(z = x+y). Z_0_v = 0 (perfect).
- **Capacity**: I(Z;X) = 0.5, I(Z;Y|X) = 1.0, I(Z;X,Y) = 1.5.

BEMAC serves as the primary development and testing channel because: - Discrete 3-symbol output makes embedding trivial (lookup table) - Perfect V-conditional channel simplifies design - Analytical results are exact (no approximation)

2.2 Additive Binary Noise MAC (ABNMAC)

Z = (X xor E_x, Y xor E_y) where (E_x, E_y) are correlated binary noise with joint distribution P(E_x, E_y). Output alphabet is {0,1} x {0,1}.

2.3 Gaussian MAC (GMAC)

The primary channel of interest for practical applications:

Z = (1 - 2X) + (1 - 2Y) + W

where X, Y in {0,1} are encoded with BPSK modulation (0 -> +1, 1 -> -1), and W ~ N(0, sigmaˆ2) is additive white Gaussian noise. The channel output Z is a continuous real value.

- **U marginal channel**: Z|X is a Gaussian mixture (not a simple AWGN channel) because Y is unknown.
- **V conditional channel** (given X): Z|Y,X is a simple AWGN channel with known mean.
- **At SNR = 6 dB** (sigmaˆ2 = 0.251): I(Z;X) = 0.464, I(Z;Y|X) = 0.912, I(Z;X,Y) = 1.376.

The GMAC is significantly harder than BEMAC for neural decoders because: 1. Continuous channel output requires a learned embedding (vs. discrete lookup) 2. The U marginal channel is a Gaussian mixture, not a clean AWGN 3. Bhattacharyya bounds and Gaussian Approximation (GA) density evolution give loose results for non-extreme paths

---

3. Polar Code Construction and Design

3.1 Encoder

The polar encoder applies the transformation X = U * B_N * Fˆ{otimes n} (mod 2), where: - B_N is the bit-reversal permutation matrix - F = [[1,0],[1,1]] is the 2x2 kernel - Fˆ{otimes n} is the n-th Kronecker power

Implementation uses the butterfly structure: first apply bit-reversal permutation, then n stages of XOR butterfly operations. Complexity: O(N log N). Batch encoding is vectorized with NumPy for efficiency.

Both users encode independently: X = polar_encode(U), Y = polar_encode(V).

3.2 Frozen Set Design

The frozen set determines which bit positions carry information vs. frozen (known) bits. Good design is critical for performance.

Analytical Methods: - **Bhattacharyya recursion**: Zˆ-(W) = 2Z - Zˆ2 (bad split), Zˆ+(W) = Zˆ2 (good split). Starting from the base channel's Bhattacharyya parameter Z_0, recursively compute all N synthetic channel parameters. Sort by Z to identify the ku best U channels and kv best V channels. - **Gaussian Approximation (GA)**: Tracks the LLR mean mu through polarization stages using phi/phi_inv functions (Trifonov 2012). Tighter than Bhattacharyya for Gaussian channels.

Critical finding: GA design assumes the extreme path 0ˆN 1ˆN (Class C). For Class B (interleaved path), GA gives **wrong frozen sets** that lead to catastrophic BLER (70%+ vs expected 5%). Monte Carlo genie-aided design must be used for Class B.

Monte Carlo Design: Run the SC decoder with a genie (true information bits provided) and count errors per synthetic channel over many trials. Channels with low error rates are selected as information positions. This accounts for the actual decoding path and gives accurate designs for all path types.

Pre-computed MC designs are stored as .npz files for block lengths N = 8 to 2048 at multiple SNR values.

3.3 Decoding Paths and Code Classes

The MAC SC decoder processes 2N leaf positions following a path b in {0,1}ˆ{2N}, where b[t] = 0 means decode a U bit and b[t] = 1 means decode a V bit.

Three canonical path types:

Class	Path	Description	CalcParent needed
A	1ˆN 0ˆN	All V first, then all U	No (extreme)
C	0ˆN 1ˆN	All U first, then all V	No (extreme)
B	0ˆ{N/2} 1ˆN 0ˆ{N/2}	Interleaved	Yes (many positions)

Class B achieves the symmetric rate point R_u = R_v = I(Z;X,Y)/2, which is the most challenging because: 1. CalcParent operations are needed at approximately N/2 tree positions 2. The interleaved path creates dependencies between U and V decisions 3. Information from both users must be combined during the tree walk

---

4. Analytical Decoders

4.1 SC Decoder

The SC decoder walks the polar code factor graph tree, computing probabilities at each node:

- **CalcLeft (f-node)**: Given parent edge and right child edge, compute left child edge. Implements the "check node" operation (circular convolution of probability tensors).
- **CalcRight (g-node)**: Given parent edge, left child edge, and left child's decision, compute right child edge. Implements the "bit node" operation.
- **CalcParent**: Given left and right child edges, compute parent edge. Required for intermediate paths when the decoder must move UP the tree.

For the MAC with joint (u,v) output, each edge carries a 2x2 probability tensor P(u,v|zˆN, decoded_bits). Operations on these tensors are: - CalcLeft: circular convolution of 2x2 tensors - CalcRight: element-wise product and normalization - CalcParent: inverse circular convolution

Implementation: Three decoder backends are auto-dispatched: 1. **Reference decoder** (`decoder.py`): O(Nˆ2) for any path, tensor-based 2. **Efficient decoder** (`efficient_decoder.py`): O(N log N) for extreme paths only 3. **Interleaved decoder** (`decoder_interleaved.py`): O(N log N) for ALL monotone chain paths (Ren et al. 2025)

4.2 SCL Decoder

The SC List decoder maintains L candidate paths through the tree walk. At each non-frozen leaf, each candidate is forked into up to 4 new candidates (one per joint (u,v) outcome). The L best candidates (by cumulative log-probability) are kept.

SCL with L >= 4 provides significant BLER improvement over SC, especially at moderate N:

N	SC BLER	SCL(L=4)	SCL(L=32)
32	0.046	0.026	0.026
64	0.025	0.013	0.012
128	0.016	0.008	0.006
256	0.005	0.0005	–
512	0.001	0.000	–

(GMAC, SNR = 6 dB, Class B, MC design)

4.3 Batch Decoding and Performance

A vectorized batch decoder (`decode_batch` with `vectorized=True`) processes multiple codewords in a tight loop, eliminating Python overhead.

Bug discovered and fixed (Session 6): The batch decoder unconditionally cast channel outputs to `int32`, destroying GMAC's continuous float values. This caused 70-95% BLER. Fixed by adding a dedicated `gaussian_mac` branch preserving `float64`.

---

5. Neural Decoder Architecture

5.1 Design Philosophy

The neural decoder replaces ALL analytical tree operations with learned neural networks. The architecture mirrors the SC decoder's computational graph:

1. **Channel Embedding** (replaces W(z|x,y) computation): Maps channel output z to a d-dimensional embedding vector.
2. **NeuralCalcLeft** (replaces circular convolution): MLP that takes parent and right-child embeddings and produces left-child embedding.
3. **NeuralCalcRight** (replaces conditional product): MLP that takes parent and left-child embeddings and produces right-child embedding.

4. **NeuralCalcParent** (replaces inverse convolution): Gated residual MLP that combines left and right child embeddings into parent embedding.
5. **Emb2Logits** (replaces probability extraction): MLP mapping d-dim embedding to 4-class log-probabilities for joint (u,v) decision.
6. **Logits2Emb** (replaces probability re-embedding): MLP mapping 4-class log-probabilities back to d-dim embedding after a decision is made.

All operations are **weight-shared** — the same MLP handles every tree node regardless of position or depth. This makes the model N-independent: a model trained at one N can decode at any N.

5.2 Architecture Details

Embedding dimensions: d = 16, hidden = 64 for baseline (39K parameters)

CalcLeft/CalcRight MLPs: - Input: concat(parent_first_half, parent_second_half, sibling) = 3d dimensions - Architecture: Linear(3d, hidden) -> ELU -> Linear(hidden, hidden) -> ELU -> Linear(hidden, d) - Output: d dimensions

NeuralCalcParent (Gated Residual):

```
gate = sigmoid(Linear(2d -> hidden) -> ELU -> Linear(hidden -> d))
candidate = MLP(2d -> hidden -> hidden -> d)
residual = (left + right) / 2
output = gate * candidate + (1 - gate) * residual
```

The residual connection is critical: it provides a stable starting point during training and ensures gradient flow through deep trees.

Parent edge structure: The parent edge embedding has size 2d, split into first half (from CalcParent) and second half (linear transform of right child). This mirrors the polar code structure where the parent carries information from both children.

5.3 GMAC Adaptation

For the Gaussian MAC, the discrete embedding `nn.Embedding(3, d)` is replaced with a continuous z_encoder:

```
z_encoder = Linear(1, z_hidden=32) -> ELU -> Linear(z_hidden, d)
```

The z_encoder maps each continuous channel output to a d-dimensional embedding independently per position. After embedding, bit-reversal permutation is applied to match the tree structure.

5.4 Tree Walk During Inference

During inference, the decoder follows the path b step by step: 1. Navigate the tree to the target leaf (using CalcLeft/CalcRight/CalcParent to compute embeddings along the way) 2. At the leaf: combine top-down embedding with bottom-up embedding, apply Emb2Logits to get 4-class log-probabilities 3. Make a hard decision (argmax for non-frozen, fixed value for frozen positions) 4. Set the leaf embedding to reflect the decision (partially deterministic tensor) 5. Move to the next leaf

This sequential process has O(N log N) steps, and during training, gradients must backpropagate through ALL steps.

6. Training Methodology

6.1 Sequential Training (Original Approach)

The model processes one codeword at a time through the full SC tree walk: - Forward pass: sequential tree walk, collecting logits at non-frozen positions - Loss: cross-entropy between predicted 4-class logits and true (u,v) targets - Teacher forcing: during training, the TRUE bit decisions are used (not the model's own estimates) - Backward pass: gradients flow through the ENTIRE sequential chain

Gradient depth: O(N log N) — at N=256, this is ~2048 sequential operations. At N=1024, ~10240 operations. This is the fundamental scalability bottleneck.

6.2 Curriculum Learning

Training from scratch at N >= 32 fails (loss stays at random). The solution is curriculum learning: 1. Train at N = 16 until convergence 2. Load weights, fine-tune at N = 32 3. Continue to N = 64, 128, 256, ...

Each stage inherits the weight-shared tree operations from the previous stage. The operations must generalize across tree depths — this is where the approach eventually fails at large N.

6.3 Knowledge Distillation

Three-phase training with an analytical CalcParent teacher: - **Phase A**: Train NeuralCalcParent with MSE supervision against analytical CalcParent output (distill_alpha = 1.0) - **Phase B**: Gradually reduce distillation weight (distill_alpha: 1.0 -> 0.0) - **Phase C**: Fine-tune all parameters jointly with pure task loss (distill_alpha = 0)

6.4 Learning Rate Schedule

Critical finding (Session 6): The original 48-hour training used cosine annealing with warm restarts, which disrupted learning by periodically jumping the learning rate back to its initial value. Switching to a **stable cosine decay** (single decay from LR to 0.01*LR, no restarts) significantly improved results: - N=128: 0.027 -> 0.019 BLER (1.69x -> 1.17x SC) - N=256: 0.033 -> 0.019 BLER (6.5x -> 3.7x SC)

6.5 Fast-CE Training (NPD-inspired, explored but not adopted)

Inspired by Aharoni et al.'s Neural Polar Decoder (NPD), we explored parallel teacher-forced training: - Process ALL N positions at each tree depth simultaneously - Use TRUE bits for BitNode conditioning (teacher forcing) - Compute cross-entropy loss at every depth level - Gradient depth reduces to O(log N) regardless of N

Results for MAC: The 4-class joint fast_ce was implemented but the loss plateaued at ~0.30 (barely below random = 1.39). The MAC's joint (u,v) structure doesn't decompose cleanly through the binary sign-flip architecture used in the single-user NPD. A two-decoder binary approach was also tried but is information-theoretically impossible at Class B rate points where R_u > I(Z;X).

7. Results

7.1 BEMAC Results

The neural decoder achieves excellent results on BEMAC, matching or beating the analytical SC decoder at all tested N:

Pure Neural CalcParent, Class B (R_u = 0.50, R_v = 0.70):

N	NN-SC BLER	SC BLER	Ratio
16	0.013	0.012	1.08x
32	0.008	0.010	**0.75x**
64	0.003	0.006	**0.50x**
128	0.001	0.002	**0.50x**
256	0.00004	0.00008	**0.50x**
512	0	0	1.0x
1024	0.0001	0.0001	1.0x

Neural SCL(L=4) vs Analytical SCL(L=4), BEMAC Class B:

N	NN-SCL(L=4)	SCL(L=4)	Improvement
32	0.007	0.008	**1.1x better**
64	0.0007	0.006	**8.6x better**
128	0.0007	0.002	**2.9x better**

The BEMAC neural decoder is a clear success story — it matches or beats the analytical decoder across all N.

7.2 GMAC Results (The Challenge)

GMAC, SNR = 6 dB, Class B (R_u approx R_v approx 0.48):

N	SC	SCL(L=4)	NN-SC (d=16)	NN-SCL(L=4)
32	0.046	0.026	0.056	**0.022**
64	0.025	0.013	0.026	**0.013**
128	0.016	0.008	0.023	0.015
256	0.005	0.0005	0.020	0.026
512	0.001	0.000	0.045	0.045
1024	0.001	0.000	0.069	0.045

Key observations: 1. NN-SCL(L=4) beats analytical SCL(L=4) at N <= 64 — this is the main positive result 2. NN-SC nearly matches SC at N = 64 (1.03x) after 48hr training 3. Catastrophic degradation at N >= 256: 4x-69x worse than SC 4. SCL(L=4) is essentially perfect at N >= 512 (zero errors)

7.3 Continued Training Results (Session 6)

Stable cosine decay LR (no warm restarts), d=16:

N	Before	After	SC	Ratio
128	0.027 (1.69x)	**0.019 (1.17x)**	0.016	Best-ever N=128
256	0.033 (6.5x)	**0.019 (3.7x)**	0.005	Quick initial gain, then plateau

7.4 d=32 Model Results

Larger model with deep residual z_encoder (157K params vs 25K baseline):

N	d=32 Best	d=16 Best	SC	Note
32	**0.046**	0.056	0.046	d=32 matches SC
64	0.046	**0.026**	0.025	d=32 worse (under-trained)
128	BLER=1.0	0.023	0.016	d=32 curriculum transfer failed

Finding: More parameters helps at N=32 but doesn't solve the N-scaling problem. The d=32 model needs disproportionately more training at each N.

7.5 Per-Level Results

Separate CalcLeft/CalcRight MLPs per tree level (189K params):

N	Per-level Best	Shared d=16	SC
128	0.056 (3.5x)	0.019 (1.2x)	0.016

Key finding: Per-level operations solve the curriculum transfer problem (start at BLER=0.265 vs shared model's 1.0 at N=128), confirming that different tree depths need different transformations. However, per-level is slower to converge and hasn't beaten the shared model yet.

---

8. Analysis: Why Neural Decoders Fail at Large N

8.1 Error Accumulation

The weight-shared CalcLeft/CalcRight MLPs introduce a small approximation error at each tree node. With d = 16, the per-node error is small but accumulates over log2(N) tree levels. At N = 32 (5 levels), the accumulated error is tolerable. At N = 256 (8 levels), it dominates.

This is analogous to the "vanishing/exploding gradient" problem in deep networks — except here the issue is accumulated inference error, not gradient flow.

8.2 GMAC vs BEMAC: Why the Gap

The neural decoder works well for BEMAC at all N but fails for GMAC at N >= 128. The key differences:

1. **Channel embedding quality**: BEMAC uses a perfect 3-symbol lookup table. GMAC uses a 2-layer MLP that must learn to encode a continuous value — this introduces an input bottleneck.
2. **Channel noise**: BEMAC is noiseless (Z = X + Y exactly). GMAC has Gaussian noise, making each channel observation inherently ambiguous.
3. **Marginal channel structure**: GMAC's U marginal channel is a Gaussian mixture, which is harder to embed than BEMAC's simple weighted sum.

8.3 Training Methodology Bottleneck

The sequential training approach requires backpropagation through O(N log N) operations. At N = 256, this means gradients flow through ~2048 sequential steps. This causes: - Gradient vanishing at early tree levels - Extremely slow convergence - Inability to learn fine-grained adjustments at deep tree levels

8.4 Comparison with NPD (Aharoni et al.)

The Neural Polar Decoder (NPD) for single-user channels achieves good results at N = 1024 using: 1. **Parallel teacher-forced training (fast_ce)**: Gradient depth O(log N) instead of O(N log N) 2. **Binary output**: Simple sign-flip encoding for g-node 3. **Residual connections**: BitNode output = MLP + analytical approximation 4. **Multi-depth loss**: CE computed at every tree depth, not just leaves

Our attempt to adapt fast_ce to the MAC setting failed because: - The 4-class joint (u,v) structure doesn't decompose through binary sign flips - Two-decoder binary approach is impossible for Class B (R_u > marginal capacity) - The MAC requires joint processing of both users, not independent per-user decoding

---

9. Things That Worked

1. **Analytical SC/SCL decoders**: Robust baseline, scales to N = 1024 with zero errors at practical rates
2. **Neural decoder for BEMAC**: Matches/beats SC at all N, including Neural SCL beating SCL(L=4) by 8x at N=64
3. **Curriculum learning**: Essential for training — from-scratch fails at N >= 32
4. **Knowledge distillation**: Enables initial CalcParent training with analytical teacher
5. **Gated residual CalcParent**: Stable training and good gradient flow
6. **Stable cosine LR decay**: Simple change from warm restarts improved N=128 from 1.69x to 1.17x SC
7. **MC design for Class B**: Correct frozen sets are critical — GA design gives wrong results for interleaved paths
8. **Batch vectorized decoding**: 30-40x speedup with NumPy, critical for large-scale evaluation

10. Things That Failed

1. **Neural GMAC at N >= 256**: Architectural limit, not training budget
2. **d=32 larger model**: More params doesn't solve N-scaling; needs disproportionately more training
3. **Fast-CE for 4-class MAC**: Loss plateaus at near-random for joint (u,v) prediction
4. **Two-decoder binary approach**: Information-theoretically impossible for Class B rates
5. **Knowledge distillation variants**: No improvement over vanilla training for GMAC
6. **BEMAC-to-GMAC transfer**: Frozen tree weights from BEMAC fail at N >= 64 for GMAC
7. **LLR front-end, FiLM conditioning**: Faster convergence but same BLER ceiling
8. **Weighted loss, label smoothing**: Hurt or no effect
9. **Per-level ops**: Solve curriculum transfer but converge too slowly to beat shared model

---

11. Current State and Open Problems

11.1 Where We Are

Channel	N <= 64	N = 128	N >= 256
BEMAC	**Solved** (beats SC)	**Solved**	**Solved**
GMAC	**Nearly solved** (1.03x SC)	**Close** (1.17x SC)	**Open** (3.7x+ SC)

11.2 Open Problems

1. **GMAC N >= 256**: The core unsolved problem. Weight-shared tree operations accumulate errors through 8+ tree levels. Need either:
   - An architecture that doesn't accumulate errors (e.g., attention across levels)
   - A training method that provides O(log N) gradient depth for the joint MAC structure
   - Per-level operations with sufficient training budget
2. **Adapting NPD's fast_ce to MAC**: The single-user NPD works because the g-node has a natural sign-flip structure. The MAC's joint 2x2 circular convolution doesn't factor as cleanly. A correct MAC-adapted parallel training method is needed.
3. **Class B interleaved decoding with neural networks**: The interleaved path creates coupling between U and V decisions that neither independent per-user decoders nor simple 4-class joint decoders handle well.
4. **Scaling training budget**: The d=16 model benefits from more training (48hr -> better results), but diminishing returns set in. The relationship between training budget, model capacity, and achievable BLER needs systematic study.

---

12. Codebase Overview

12.1 Core Library (`polar/`)

Module	Purpose	Complexity
`encoder.py`	Polar encoder (bit-reversal + butterfly)	O(N log N)
`channels.py`	BEMAC, ABNMAC, GaussianMAC	O(1) per sample
`design.py`	Bhattacharyya + GA density evolution	O(N)
`design_mc.py`	Monte Carlo genie-aided design	O(N * trials)
`decoder.py`	Unified SC decoder (auto-dispatch)	O(N log N)
`decoder_scl.py`	SC List decoder	O(L * N log N)
`decoder_interleaved.py`	SC for all monotone chain paths	O(N log N)
`eval.py`	BER/BLER Monte Carlo evaluation	configurable

12.2 Neural Decoders (`neural/`)

Module	Architecture	Params	Channel
`ncg_pure_neural.py`	Gated residual CalcParent	39K	BEMAC
`ncg_gmac.py`	+ continuous z_encoder	25K	GMAC
`neural_scl.py`	Neural SCL (list decode on NN)	same	both
`train_gmac_d32.py`	d=32, deep z_encoder	157K	GMAC
`train_gmac_perlevel.py`	Per-level CalcLeft/Right	189K	GMAC

12.3 Pre-computed Resources

- **Designs**: MC frozen sets for GMAC Classes A/B/C at SNR 0-10 dB, N = 8 to 2048
- **Saved models**: Trained checkpoints for N = 32 to 1024
- **Results**: 824+ simulation data points across channels, classes, decoders

---

13. References

1. E. Arikan, "Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels," IEEE Trans. Inf. Theory, vol. 55, no. 7, pp. 3051-3073, Jul. 2009.
2. S. B. Onay, "Successive cancellation decoding of polar codes for the two-user binary-input MAC," Proc. IEEE ISIT, pp. 1532-1536, Jul. 2013.
3. Y. Ren, Z. Li, and P. M. Olmos, "Successive cancellation decoding of polar codes for the two-user MAC using computational graphs," in preparation, 2025.
4. S. Aharoni, R. Misoczki, and E. Ordentlich, "Neural polar decoders for 5G: An industrial perspective," IEEE J. Sel. Areas Commun., 2024.
5. I. Tal and A. Vardy, "List decoding of polar codes," IEEE Trans. Inf. Theory, vol. 61, no. 5, pp. 2213-2226, May 2015.

Appendix A: Key Hyperparameters

Parameter	BEMAC baseline	GMAC baseline	d=32 variant
d (embedding)	16	16	32
hidden	64	64	128
n_layers	2	2	2
z_hidden	N/A	32	64
Learning rate	1e-3	1e-4 to 8e-5	3e-4
Batch size	64	8-16	8-32
Optimizer	AdamW	AdamW	AdamW
Weight decay	1e-5	1e-5	1e-5
Grad clipping	1.0	1.0	1.0

Appendix B: Complete GMAC BLER Table (SNR = 6 dB, Class B)

N	SC	SCL(4)	SCL(32)	NN-SC	NN-SCL(4)	d=32	Per-level
32	0.046	0.026	0.026	0.056	0.022	0.046	0.053
64	0.025	0.013	0.012	0.026	0.013	0.046	0.048
128	0.016	0.008	0.006	0.019*	0.015	failed	0.056
256	0.005	0.0005	–	0.019*	0.026	–	–
512	0.001	0.000	–	0.045	0.045	–	–
1024	0.001	0.000	–	0.069	0.045	–	–

*After continued training with stable LR schedule (Session 6 improvement)